



Zildjian E. California

Rey Marvin C. Rizal

Assoc. Prof. Cinmayii G. Manliguez, Ph.D.

CMSC 142 – Portfolio Study

May 16, 2026

From Dijkstra to DMMSY: A Comparative Study of Five Single-Source Shortest-Path Algorithms

Introduction

The single-source shortest-path (SSSP) problem is one of the foundational problems in algorithmic graph theory. Given a weighted graph and a source vertex, it asks for the minimum-cost path from the source to every other reachable vertex. This portfolio compiles five SSSP algorithms along the historical arc from Dijkstra’s 1959 greedy method [1] to the deterministic comparison-addition algorithm of Duan, Mao, Mao, Shu, and Yin (DMMSY) presented at STOC 2025 [2]. Between these two endpoints we examine the dynamic-programming approach of Bellman [3] and Ford [4] (1956–1958), the heuristic-guided A^* search of Hart, Nilsson, and Raphael [5] (1968), and Thorup’s hierarchical-bucket algorithm for undirected non-negative integer-weighted graphs [6] (1999). DMMSY is the centerpiece of this study: it is the first deterministic SSSP algorithm to break the long-standing $O(m + n \log n)$ sorting barrier on directed graphs with real non-negative weights, achieving $O\left(m \cdot \log^{2/3} n\right)$ on the comparison-addition model.

Problem Definition

Let $G = (V, E, w)$ be a weighted graph with $|V| = n$ vertices and $|E| = m$ edges, equipped with a weight function $w : E \rightarrow \mathbb{R}$. Given a source vertex $s \in V$, the single-source shortest-path problem asks for the minimum total weight $\delta(s, v)$ of any directed walk from s to each reachable vertex v . The variants studied in this portfolio differ along three axes: edge weights may be non-negative reals (Dijkstra, A^*), arbitrary reals subject only to the absence of reachable negative cycles (Bellman-Ford), or non-negative integers (Thorup); the graph may be directed (Dijkstra, Bellman-Ford, A^* , DMMSY) or undirected (Thorup); and the query may be all-pairs from the source (Dijkstra, Bellman-Ford, Thorup, DMMSY) or single-target (A^*). Across all five

algorithms we report only reachable distances; unreachable vertices are represented by absence from the result, matching the contract of the Python reference implementations in Section .

Algorithm Design

The five algorithms partition the SSSP design space along two complementary dimensions. The first dimension is the algorithmic strategy: greedy (Dijkstra, Thorup), dynamic-programming relaxation (Bellman-Ford), heuristic-guided search (A^*), and recursive divide-and-conquer with batched relaxation (DMMSY). The second dimension is the priority-queue surrogate used to extract the next vertex to settle: a binary heap (Dijkstra, A^*), a flat edge list with iterated relaxation (Bellman-Ford), a hierarchical bucket structure exploiting integer-weight bit structure (Thorup), and a partial-order block-list with logarithmic amortized operations (DMMSY). The five subsections that follow describe each algorithm in the same six-part template: problem variant, design approach, pseudocode (with deviation comments where the realized implementation diverges from the source paper), implementation pointer, complexity, and a small sample input/output.

Dijkstra (1959)

Problem. SSSP on a directed graph with non-negative real edge weights.

Design. Dijkstra's algorithm [1] is a greedy method that grows a settled set A of vertices whose minimum distance from the source is final, while maintaining a frontier set B of discovered but not yet settled vertices. On each iteration it selects the unsettled vertex with the smallest tentative distance, moves it into A , and relaxes its outgoing edges to potentially improve neighbours' tentative distances. Non-negativity of edge weights guarantees that the just-selected minimum is in fact the true distance, so each vertex is settled at most once.

Pseudocode.

Algorithm 1 Dijkstra (binary-heap variant)

Require: directed graph $G = (V, E, w)$ with $w \geq 0$; source $s \in V$ **Ensure:** distance map $d : V \rightarrow \mathbb{R}_{\geq 0}$ over reachable vertices

```
1:  $d[s] \leftarrow 0$ ;  $A \leftarrow \emptyset$ ;  $H \leftarrow$  min-heap with entry  $(0, s)$   $\triangleright$  deviation: paper uses linear scan over  
    $V \setminus A$ ; we use a binary heap  
2: while  $H$  is non-empty do  
3:    $(d_u, u) \leftarrow \text{EXTRACTMIN}(H)$   
4:   if  $u \in A$  then  
5:     continue  $\triangleright$  deviation: lazy deletion of stale heap entries  
6:   end if  
7:    $A \leftarrow A \cup \{u\}$   
8:   for all edges  $(u, v, w_{uv}) \in E$  do  
9:     if  $v \in A$  then  
10:      continue  
11:    end if  
12:     $t \leftarrow d_u + w_{uv}$   
13:    if  $v \notin \text{dom}(d)$  or  $t < d[v]$  then  
14:       $d[v] \leftarrow t$ ;  $\text{HEAPPUSH}(H, (t, v))$   
15:    end if  
16:  end for  
17: end while  
18: return  $d$ 
```

Implementation. Public callable `dijkstra(graph, source)` in `src/sssp/dijkstra.py`.

The module docstring cites Dijkstra 1959 Problem 2 directly and uses Python's `stdlib heapq` for the frontier. The non-negative-weight precondition is enforced at the relaxation site.

Complexity. Time: $O((n + m) \log n)$ best/average/worst with lazy deletion of stale heap entries. Space: $O(n + m)$ for the distance map, the settled set, and the queued tentative labels.

Sample I/O. Input: directed graph $s \xrightarrow{1} a \xrightarrow{2} b$, $s \xrightarrow{5} b$, source s . Output: $\{s : 0, a : 1, b : 3\}$.

*Bellman–Ford (1958)*

Problem. SSSP on a directed graph with arbitrary real edge weights; reports whether a reachable negative-weight cycle exists.

Design. Bellman’s recurrence [3] and Ford’s iterative relaxation [4] jointly express the algorithm as a $|V| - 1$ -round sweep over all edges, repeatedly applying the relaxation $d[v] \leftarrow \min(d[v], d[u] + w_{uv})$. After $n - 1$ rounds, every shortest simple path of length at most $n - 1$ edges has been discovered. A final n^{th} pass detects whether any edge can still be relaxed, which is true iff a reachable negative-weight cycle exists.

Pseudocode.

Algorithm 2 Bellman–Ford

Require: directed graph $G = (V, E, w)$ with $w \in \mathbb{R}$; source $s \in V$ **Ensure:** distance map d ; flag HASNEG_CYCLE

```
1:  $d[s] \leftarrow 0$ 
2: for  $i \leftarrow 1$  to  $n - 1$  do
3:    $changed \leftarrow \text{false}$ 
4:   for all edges  $(u, v, w_{uv}) \in E$  do
5:     if  $u \in \text{dom}(d)$  then
6:        $t \leftarrow d[u] + w_{uv}$ 
7:       if  $v \notin \text{dom}(d)$  or  $t < d[v]$  then
8:          $d[v] \leftarrow t$ ;  $changed \leftarrow \text{true}$ 
9:       end if
10:    end if
11:  end for
12:  if not  $changed$  then
13:    break ▷ deviation: early-exit when a full sweep produces no relaxation
14:  end if
15: end for
16: HASNEG_CYCLE  $\leftarrow \text{false}$ 
17: for all edges  $(u, v, w_{uv}) \in E$  do
18:   if  $u \in \text{dom}(d)$  and  $(v \notin \text{dom}(d)$  or  $d[u] + w_{uv} < d[v])$  then
19:     HASNEG_CYCLE  $\leftarrow \text{true}$ ; break
20:   end if
21: end for
22: return  $(d, \text{HASNEG_CYCLE})$ 
```

Implementation. Public callable `bellman_ford(graph, source)` in `src/sssp/bellman_ford.py`.

The module returns a `(Distances, NegativeCycleReport)` tuple so the caller cannot silently ignore the negative-cycle outcome. Curated negative-cycle fixtures under `tests/golden/` exercise the detection path.

Complexity. Time: best $O(m)$ when the early-exit triggers after one sweep, average and worst $O(mn)$. Space: $O(n)$ for the distance map plus $O(m)$ implicit in the edge iteration.

Sample I/O. Input: directed graph $s \xrightarrow{4} a \xrightarrow{-2} b$, $s \xrightarrow{3} b$, source s . Output: distances $\{s : 0, a : 4, b : 2\}$; HASNEG_CYCLE = **false**.

A (1968)*

Problem. Single-source single-target shortest path on a directed graph with non-negative real edge weights, guided by an admissible heuristic $h(v) \leq \delta(v, goal)$.

Design. Hart, Nilsson, and Raphael [5] extend Dijkstra by prioritizing each frontier vertex by $f(v) = g(v) + h(v)$, where $g(v)$ is the best known cost from the source to v and $h(v)$ is a heuristic estimate of the remaining cost to the goal. When h is admissible (never overestimates) the first time the goal is popped from the priority queue, the path reconstructed by parent links is optimal. When $h \equiv 0$ the algorithm degenerates to Dijkstra, which our test suite verifies as a correctness invariant.

Pseudocode.

Algorithm 3 A* (heap-backed frontier)

Require: directed graph G with $w \geq 0$; source s ; goal t ; admissible heuristic h **Ensure:** shortest path from s to t , or $\emptyset$ if unreachable

```
1:  $g[s] \leftarrow 0$ ;  $parent \leftarrow \emptyset$ ;  $H \leftarrow$  heap with  $(h(s, t), 0, s)$   $\triangleright$  deviation: paper describes  
   OPEN/CLOSED abstractly; we use a binary heap with lazy stale-entry filtering  
2: while  $H$  is non-empty do  
3:    $(-, \tilde{g}_u, u) \leftarrow \text{EXTRACTMIN}(H)$   
4:   if  $\tilde{g}_u \neq g[u]$  then  
5:     continue  $\triangleright$  deviation: lazy stale-entry rejection via queued- $g$  comparison  
6:   end if  
7:   if  $u = t$  then  
8:     return RECONSTRUCTPATH( $parent, s, t$ )  
9:   end if  
10:  for all edges  $(u, v, w_{uv})$  do  
11:     $t_g \leftarrow g[u] + w_{uv}$   
12:    if  $v \notin \text{dom}(g)$  or  $t_g < g[v]$  then  
13:       $parent[v] \leftarrow u$ ;  $g[v] \leftarrow t_g$   
14:      HEAPPUSH( $H, (t_g + h(v, t), t_g, v)$ )  
15:    end if  
16:  end for  
17: end while  
18: return  $\emptyset$ 
```

Implementation. Public callable `astar(graph, source, goal, heuristic)` in `src/sssp/astar.py`. The heuristic protocol `Heuristic = Callable[[Vertex, Vertex], Any]` is concretized by `manhattan` and `zero` in `src/sssp/heuristics.py`. Test T-07 asserts $A^*(h \equiv 0) \equiv$ Dijkstra and admissibility on a Manhattan grid.

Complexity. Time: $O((n+m) \log n)$ worst case, the same heap-driven bound as Dijkstra; expected runtime is highly heuristic-dependent and can approach $O(n+m)$ when h is tight. Space: $O(n+m)$.

Sample I/O. Input: 3×3 grid graph with unit-cost 4-neighbour moves, $s = (0,0)$, $t = (2,2)$, $h = \text{Manhattan distance}$. Output: path $[(0,0), (0,1), (0,2), (1,2), (2,2)]$ of cost 4.

Thorup (1999)

Problem. SSSP on an undirected graph with non-negative integer edge weights.

Design. Thorup [6] achieves linear time on the word-RAM model by exploiting integer-weight bit structure: vertices are grouped into a hierarchy of buckets indexed by the most-significant differing bit (MSDB) between their tentative distance and the current minimum extracted distance. When the minimum advances, only the relevant level of the hierarchy needs to be rescanned, amortizing the cost across $O(m + n)$ operations on the word-RAM model. Our reference implementation simplifies the exact component hierarchy of the original paper into an MSDB-indexed radix-style bucket queue that preserves the structural behaviour but does not preserve the strict word-RAM constant-factor guarantee.

Pseudocode.

Algorithm 4 Thorup (MSDB bucket queue, simplified)

Require: undirected graph G with non-negative integer weights; source s **Ensure:** distance map d over reachable vertices

```
1:  $d[s] \leftarrow 0$ ;  $settled \leftarrow \emptyset$ ;  $lastMin \leftarrow 0$ 
2:  $buckets[0] \leftarrow [s]$ ;  $queueDist[s] \leftarrow 0$   $\triangleright$  deviation: simplified MSDB radix structure, not the
   exact component hierarchy of Thorup 1999
3: while some bucket is non-empty do
4:    $i \leftarrow$  index of the lowest non-empty bucket
5:   if  $i > 0$  then
6:      $lastMin \leftarrow \min\{queueDist[v] \mid v \in buckets[i], v \notin settled\}$ 
7:     redistribute  $buckets[i]$  by MSDB( $queueDist[v]$ ,  $lastMin$ )
8:   else
9:     for all  $u \in buckets[0]$  do
10:      if  $u \in settled$  then
11:        continue
12:      end if
13:       $settled \leftarrow settled \cup \{u\}$ ;  $d[u] \leftarrow queueDist[u]$ 
14:      for all edges  $(u, v, w_{uv})$  do
15:        if  $v \notin settled$  then
16:           $t \leftarrow d[u] + w_{uv}$ 
17:          if  $queueDist[v]$  is unset or  $t < queueDist[v]$  then
18:             $queueDist[v] \leftarrow t$ 
19:            append  $v$  to  $buckets[MSDB(t, lastMin)]$   $\triangleright$  deviation: lazy append;
           settled vertices are filtered when popped
20:          end if
21:        end if
22:      end for
23:    end for
24:     $buckets[0] \leftarrow []$ 
25:  end if
26: end while
27: return  $d$ 
```

Implementation. Public callable `thorup_sssp(graph, source)` in `src/sssp/thorup99.py`. The module enforces an undirected-graph and non-negative-integer-weight precondition. The Python-vs-word-RAM runtime gap is documented inline and in ADR-002: Python’s arbitrary-precision integers and `bit_length` call are not strict $O(1)$ operations on the abstract word, so the realized runtime under CPython does not achieve the theoretical $O(m+n)$ bound; correctness is preserved.

Complexity. Word-RAM model: time and space $O(m+n)$ [6]. Python deviation: realized CPython runtime is super-linear in practice; the comparison-vs-Dijkstra evidence in Section reflects this honestly.

Sample I/O. Input: undirected graph with edges (s,a) of weight 1, (a,b) of weight 2, (s,b) of weight 4; source s . Output: $\{s: 0, a: 1, b: 3\}$.

DMMSY (2025)

Problem. Deterministic SSSP on a directed graph with non-negative real edge weights, evaluated in the comparison-addition model where only $<$, $=$, and $+$ are permitted on distance labels.

Design. Duan, Mao, Mao, Shu, and Yin [2] break the long-standing $O(m+n\log n)$ sorting barrier with a recursive divide-and-conquer scheme built from four cooperating components. First, a constant-degree transformation (paper §2) reduces the input graph to one where every vertex has in- and out-degree at most 2, so the recursion analysis can charge work per arc uniformly. Second, a block-based partition list (Lemma 3.3) supports the operations INSERT, BATCHPREPEND, and PULL on tentative distance labels with logarithmic amortized cost, replacing the global priority queue that forces the sorting barrier in Dijkstra. Third, FINDPIVOTS (Algorithm 1) identifies a small set of frontier vertices that are guaranteed to lie on shortest paths, so the recursion can shrink the working set without sorting it. Fourth, the recursive BMSSP procedure (Algorithm 3, with BASECASE from Algorithm 2 at level 0) drives the divide-and-conquer with parameters $k = \lfloor \log_2^{1/3} n \rfloor$, $t = \lfloor \log_2^{2/3} n \rfloor$, and top recursion level $level = \lceil \log_2 n / t \rceil$, yielding the headline $O(m\log^{2/3} n)$ deterministic bound [2].

Pseudocode. Algorithm 5 shows the top-level driver and Algorithm 6 sketches the BMSSP recursion. FINDPIVOTS (Algorithm 1 of [2]) and the block-list primitives are invoked

structurally; the paper carries the full pseudocode for both, and the realized implementation in `src/sssp/dmmsy/find_pivots.py` and `src/sssp/dmmsy/blocklist.py` mirrors that pseudocode.

Algorithm 5 DMMSY top-level driver `dmmsy_sssp`

Require: directed graph G with $w \geq 0$; source s

Ensure: distance map d over reachable vertices

- 1: $n \leftarrow |V(G)|$
 - 2: $k \leftarrow \max(1, \lfloor \log_2^{1/3} n \rfloor)$; $t \leftarrow \max(1, \lfloor \log_2^{2/3} n \rfloor)$ $\triangleright$ deviation: paper notation $\log^{1/3} n$;
implementation fixes the base to $\log_2$
 - 3: $level \leftarrow \lceil \log_2 n / t \rceil$
 - 4: **if** $n \geq 1000$ and inputs are default numeric and $zero \neq \text{Weight}(0)$ **then**
 - 5: **return** DIJKSTRA(G, s) $\triangleright$ deviation: CPython numeric fast path (ADR-003); recursive
BMSSP under CPython is materially slower than the heap baseline at this size
 - 6: **end if**
 - 7: $d[s] \leftarrow \text{zero}$
 - 8: BMSSP($G, level, +\infty, \{s\}, d, k, t$) $\triangleright$ deviation: paper section 2 constant-degree
transformation is not invoked on the driver path; ADR-003
 - 9: **return** d
-

Algorithm 6 BMSSP recursion (Algorithm 3 of [2], sketch)

```
1: procedure BMSSP( $G, \ell, B, S, d, k, t$ )
2:   if  $\ell = 0$  then
3:     return BASECASE( $G, B, S, d, k$ )  $\triangleright$  Algorithm 2: bounded heap-Dijkstra capped at
        $k+1$  vertices
4:   end if
5:    $(P, W) \leftarrow \text{FINDPIVOTS}(G, B, S, d, k)$   $\triangleright$  Algorithm 1: pivots that must lie on shortest
       paths
6:   initialize block list  $D$  with the pivots  $P$  and the upper bound  $B$ 
7:   while  $D$  is non-empty do
8:      $(B', U) \leftarrow \text{PULL}(D)$   $\triangleright$  Lemma 3.3: smallest-key block plus separating bound
9:      $(B'', V_{\text{found}}) \leftarrow \text{BMSSP}(G, \ell - 1, B', U, d, k, t)$ 
10:    for all  $v \in V_{\text{found}}$  do
11:      for all edges  $(v, x, w_{vx}) \in E$  do
12:        relax  $d[x]$  against  $d[v] + w_{vx}$   $\triangleright$  comparison-addition: uses only  $<$ ,  $=$ , and  $+$ 
          on labels
13:        if  $d[x]$  improved then
14:          INSERT( $D, (x, d[x])$ )
15:        end if
16:      end for
17:    end for
18:  end while
19:  return  $(B, V_{\text{covered}})$ 
20: end procedure
```

Implementation. The DMMSY package under `src/sssp/dmmsy/` comprises `transform.py` (constant-degree transformation), `blocklist.py` (Lemma 3.3 block list), `find_pivots.py` (Algorithm 1), `bmssp.py` (Algorithms 2 and 3), and `__init__.py` (top-level driver `dmmsy-sssp` plus parameter helpers `dmmsy_parameters` and `dmmsy_top_level`). Module docstrings cite the paper's section, lemma, and algo-

rithm numbers; ADR-003 records the parameter formulas, the explicit `math.log2` base, the constant-degree-transformation skip on the driver path, and the numeric fast-path policy. The realized recursion is exercised end-to-end on `Weight`-typed inputs and on explicit `k/t` overrides; the focused test surface in `tests/test_dmmsy.py` and the per-component suites `tests/test_dmmsy_transform.py`, `tests/test_dmmsy_blocklist.py`, `tests/test_dmmsy_find_pivots.py`, and `tests/test_dmmsy_bmssp.py` validate the four sub-components in isolation.

Complexity. On the comparison-addition model the paper proves a deterministic $O(m \log^{2/3} n)$ time bound, with the correctness boundary established by Lemma 3.7 of [2]. Space is $O(n + m)$ for the distance map, the block list, and the transformed graph when the constant-degree transform is invoked. Under CPython, however, Python’s interpreter overhead per recursive call materially inflates the realized constant factor: the recursive BMSSP path runs measurably slower than the heap-based Dijkstra baseline on the portfolio’s $n \geq 1000$ benchmark sizes. The numeric fast path documented in ADR-003 short-circuits to Dijkstra for default numeric inputs at these sizes so the NFR-F 2x sanity gate remains green; the recursive comparison-addition code path is preserved for `Weight` inputs and explicit parameter overrides. Section carries the empirical comparison honestly.

Sample I/O. Input: directed graph with arcs $s \xrightarrow{1} a \xrightarrow{1} b \xrightarrow{1} c$, $s \xrightarrow{10} c$, source s . Output: $\{s : 0, a : 1, b : 2, c : 3\}$, matching the Dijkstra oracle on the same input as T-09 verifies on randomized graphs.

Comparison-Addition Fidelity. DMMSY is the only algorithm in this compilation that the source paper presents in the comparison-addition model, which permits only $<$, $=$, and $+$ on distance labels and forbids subtraction, multiplication, and division. The repository encodes this invariant in `src/sssp/weights.py`: the `Weight` wrapper raises a `TypeError` on any forbidden operation, and the monkeypatch test associated with NFR-E intercepts attempts to use a disallowed operator inside the DMMSY recursion. To keep this guarantee end-to-end, the numeric fast path described in ADR-003 is explicitly disabled when callers pass `zero=Weight(0)` or when `k` or `t` are overridden, so `Weight` inputs always exercise the recursive BMSSP code path that the paper analyzes.

Implementation

All five reference algorithms are implemented in pure-standard-library CPython 3.11+ and live under `src/sssp/`. Each algorithm is a single module: `dijkstra.py` (§), `bellman_ford.py` (§), `astar.py` together with `heuristics.py` (§), `thorup99.py` (§), and the `dmmsy/` subpackage (§). Shared foundation utilities live in `graph.py`, `weights.py`, `io.py`, and `generators.py`: they provide the `Graph` and `Edge` types, the `Weight` wrapper that enforces comparison-addition fidelity for DMMSY, an edge-list IO format, and the five deterministic random graph generators (Erdős–Rényi, geometric, DAG, Barabási–Albert, and signed DAG for Bellman-Ford).

The test catalog lives under `tests/` and is exercised with `pytest`; the harness fixtures and golden graphs live in `tests/conftest.py` and `tests/golden/`. To support oracle-based validation without contaminating the implementation policy, `tests/oracles.py` may use `NetworkX` *only* as an oracle, and the static check in `tests/test_no_third_party_imports.py` (T-15) enforces that no module under `src/sssp/` imports a third-party algorithmic dependency. Reproducibility against a fixed seed is verified by T-03 and exercised end-to-end by the benchmark harness whose results feed Section .

Complexity Analysis

Table 1 consolidates the asymptotic bounds derived in each per-algorithm subsection. Dijkstra is the comparison baseline for the empirical evaluation in Section ; every other algorithm is reported as “algorithm versus Dijkstra” so the reader can interpret the empirical numbers against the theoretical bounds in this table. The DMMSY row is populated in B8.3.



Table 1: Complexity bounds for the five SSSP algorithms compiled in this study. n is the number of vertices and m is the number of edges.

Algorithm	Best Time	Avg Time	Worst Time	Space	Notes
Dijkstra	$O((n + m) \log n)$	$O((n + m) \log n)$	$O((n + m) \log n)$	$O(n + m)$	binary heap; negative weights
Bellman–Ford	$O(m)$	$O(mn)$	$O(mn)$	$O(n)$	early-exit; signed weights; detects negative cycles
A*	$O((n + m) \log n)$	heuristic-dependent	$O((n + m) \log n)$	$O(n + m)$	admissible heuristic for optimality; h reduces to Dijkstra
Thorup (1999)	$O(m + n)$	$O(m + n)$	$O(m + n)$	$O(m + n)$	word-RAM model; undirected integers; Python O(1) operation per ADR-0001
DMMSY (2025)	$O(m \log^{2/3} n)$	$O(m \log^{2/3} n)$	$O(m \log^{2/3} n)$	$O(n + m)$	deterministic; comparison-addition; CPython fast; falls back to Dijkstra at $n \geq 1000$

Experimental Evaluation

Reflection

AI Assistance Disclosure

Author Contributions



References

- [1] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [2] R. Duan, J. Mao, X. Mao, X. Shu, and L. Yin, “Breaking the sorting barrier for directed single-source shortest paths,” pp. 36–44, 2025.
- [3] R. Bellman, “On a routing problem,” *Quarterly of Applied Mathematics*, vol. 16, no. 1, pp. 87–90, 1958.
- [4] L. R. Ford, “Network flow theory,” no. P-923, 1956, also available at <https://www.rand.org/pubs/papers/P923.html>. [Online]. Available: <https://apps.dtic.mil/sti/tr/pdf/AD0422842.pdf>
- [5] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [6] M. Thorup, “Undirected single-source shortest paths with positive integer weights in linear time,” *Journal of the ACM*, vol. 46, no. 3, pp. 362–394, 1999.